

Ficheros sensibles escribibles

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ciertos ficheros del sistema definen quien es root y quien puede escalar. Si por una mala configuración de permisos tu usuario puede escribir en `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, puedes añadir o alterar una cuenta con uid 0 o darte permisos sudo. No hay exploit: es escritura directa a la fuente de la verdad.

Estos ficheros deberían ser solo-lectura para usuarios normales. Cuando alguien afloja los permisos (o un script los deja escribibles), rompes la frontera de confianza mas básica del sistema: `/etc/passwd` todavía permite un hash de password en el segundo campo, así que puedes crear un root con password conocida.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la enumeración (`enum-privesc-linux`) marca `/etc/passwd`, `/etc/shadow` o `/etc/sudoers` como escribibles por ti (`ls -l` o el check de linpeas).

Cómo identificarla

- `ls -l /etc/passwd` muestra permiso de escritura para tu usuario o grupo.
- `/etc/shadow` legible/escribible (linpeas lo resalta).
- Un fichero en `/etc/sudoers.d/` escribible por ti.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

`ls -l /etc/passwd /etc/shadow /etc/sudoers` y busca un permiso `w` para ti.

Evidencia esperada: Un 'w' en los permisos de alguno de esos ficheros para tu usuario o grupo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ls -l /etc/passwd /etc/shadow /etc/sudoers
```

Objetivo y contexto: Comprueba de un vistazo si tienes escritura en los ficheros que definen identidad y privilegios. Es un check barato con recompensa enorme.

Resultado esperado: Los permisos. Una 'w' para tu usuario/grupo en cualquiera de ellos es escalada directa.

Normalmente solo root escribe; si no, hay mala config.

Capa 2: prueba documentada

```
openssl passwd -1 -salt xyz Passw0rd
```

Objetivo y contexto: Genera un hash valido para /etc/passwd. Si passwd es escribible, anades una linea 'hacker:HASH:0:0:/root:/bin/bash' y creas un root con password conocida.

Resultado esperado: El hash a pegar en el segundo campo. Tras anadir la linea, 'su hacker' con Passw0rd te da uid=0. -1 usa MD5crypt, compatible con /etc/passwd.

Anatomía de los payloads

La primera prueba es `ls -l /etc/passwd /etc/shadow /etc/sudoers`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 'w' en los permisos de alguno de esos ficheros para tu usuario o grupo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba permisos de /etc/passwd, /etc/shadow y /etc/sudoers(.d).
- Si /etc/passwd es escribible, genera un hash (openssl passwd) y anade una linea de usuario con uid 0.
- Cambia a ese usuario con su y la password que pusiste.
- Confirma uid=0. Alternativa: darte una linea NOPASSWD en sudoers.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- /etc/passwd, /etc/shadow y /etc/sudoers(.d) son todos solo-lectura para tu usuario/grupo.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si puedes escribir `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, te conviertes en root editando una línea. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (writable-sensitive-files)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.